Streaming-Order AMM: Solidity source code

FRE-GY 6931, Fall 2026. Team: Aditya Shah (as22008), Siddhant Yadav (sy4583), Dev Salvi (ds8259). Network: Ethereum Sepolia.
Repository: https://github.com/Sidziesama/StreamingOrderAMM_BlockchainFinalProject (graded at tag `final`) · contracts last changed in commit `6854da9`. Every contract below is deployed on Sepolia and verified on Etherscan (addresses in the report, Section 3.5).

OpenZeppelin v5 provides ERC-20, ERC-721, Ownable, ReentrancyGuard, SafeERC20, SafeCast and Math; Chainlink's `AggregatorV3Interface` provides the feed interface. Neither is reproduced here.

1. packages/hardhat/contracts/Pool.sol (240 lines): Constant-product pool; also the SALP ERC-20 LP token
2. packages/hardhat/contracts/OrderBook.sol (186 lines): ERC-721 streaming orders (SORDER)
3. packages/hardhat/contracts/PriceGuard.sol (89 lines): Chainlink checks and pool deviation band
4. packages/hardhat/contracts/libraries/PriceMath.sol (33 lines): All price and amount conversions in one place
5. packages/hardhat/contracts/interfaces/IPool.sol (70 lines): Interface (frozen)
6. packages/hardhat/contracts/interfaces/IOrderBook.sol (58 lines): Interface (frozen)
7. packages/hardhat/contracts/interfaces/IPriceGuard.sol (36 lines): Interface (frozen)
8. packages/hardhat/contracts/mocks/MockWBTC.sol (34 lines): Test WBTC, 8 decimals, capped faucet
9. packages/hardhat/contracts/mocks/MockUSDC.sol (34 lines): Test USDC, 6 decimals, capped faucet

Pool.sol

Constant-product pool; also the SALP ERC-20 LP token · deployed and verified on Sepolia at `0x9461fd23335242663ddf7a9fae994995df7fc8e9`

```
//SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

import { ERC20 } from "@openzeppelin/contracts/token/ERC20/ERC20.sol";
import { IERC20 } from "@openzeppelin/contracts/token/ERC20/IERC20.sol";
import { IERC20Metadata } from "@openzeppelin/contracts/token/ERC20/extensions/IERC20Metadata.sol";
import { SafeERC20 } from "@openzeppelin/contracts/token/ERC20/utils/SafeERC20.sol";
import { ReentrancyGuard } from "@openzeppelin/contracts/utils/ReentrancyGuard.sol";
import { Math } from "@openzeppelin/contracts/utils/math/Math.sol";
import { SafeCast } from "@openzeppelin/contracts/utils/math/SafeCast.sol";
import { IPool } from "./interfaces/IPool.sol";

/// @title Pool
/// @notice Constant-product (x * y = k) pool for one token pair with a flat 30 bps fee, ERC-20 LP shares and a
///         TWAP accumulator. The pool contract is itself the LP share token.
/// @dev All maths is integer maths with floor division, so every number can be reproduced exactly in Python.
///      There is no owner and no admin function: nothing in this contract can be changed after deployment.
///      Fee-on-transfer and rebasing tokens are not supported.
contract Pool is IPool, ERC20, ReentrancyGuard {
    using SafeERC20 for IERC20;

    /// @notice Swap fee in basis points, taken from the input amount and left in the reserves.
    uint256 public constant override FEE_BPS = 30;
    uint256 internal constant BPS = 10_000;

    /// @notice Shares locked forever on the first deposit so the pool can never be fully drained.
    uint256 public constant MINIMUM_LIQUIDITY = 1000;
    address internal constant DEAD = 0x000000000000000000000000000000000000dEaD;

    address public immutable override token0;
    address public immutable override token1;

    /// @dev 10 ** (18 + decimals0) and 10 ** decimals1, so that
    ///      priceE18 = reserve1 * priceScaleNum / (reserve0 * priceScaleDen) is USDC per whole WBTC scaled by 1e18.
    uint256 internal immutable priceScaleNum;
    uint256 internal immutable priceScaleDen;

    /// @dev Both reserves and the last update time share ONE storage slot (112 + 112 + 32 bits), so each
    ///      state-changing call writes it once. Matches the layout `getReserves` returns.
    struct Reserves {
        uint112 reserve0;
        uint112 reserve1;
        uint32 timestamp;
    }

    Reserves internal _reserves;

    /// @notice Sum of (priceE18 * seconds) since the first deposit.
    uint256 public priceCumulative;

    struct Observation {
        uint32 timestamp;
        uint224 cumulative;
    }

    /// @dev One entry per distinct block timestamp in which reserves changed, plus an initial entry (t0, 0).
    ///      The price in effect between entries i and i+1 is (cum[i+1] - cum[i]) / (ts[i+1] - ts[i]), exactly.
    Observation[] internal observations;

    constructor(address token0_, address token1_) ERC20("Streaming AMM LP", "SALP") {
        if (token0_ == address(0) || token1_ == address(0) || token0_ == token1_) revert InvalidToken();
        token0 = token0_;
        token1 = token1_;
        priceScaleNum = 10 ** (18 + IERC20Metadata(token0_).decimals());
        priceScaleDen = 10 ** IERC20Metadata(token1_).decimals();
    }

    // ---------------------------------------------------------------- views

    /// @inheritdoc IPool
    function getReserves() external view override returns (uint112, uint112, uint32) {
        return (_reserves.reserve0, _reserves.reserve1, _reserves.timestamp);
    }

    /// @inheritdoc IPool
    function getAmountOut(address tokenIn, uint256 amountIn) public view override returns (uint256 amountOut) {
        if (amountIn == 0) revert ZeroAmount();
        if (tokenIn != token0 && tokenIn != token1) revert InvalidToken();
        (uint256 reserveIn, uint256 reserveOut) = tokenIn == token0 ? (_reserves.reserve0, _reserves.reserve1) :
(_reserves.reserve1, _reserves.reserve0);
        if (reserveIn == 0 || reserveOut == 0) revert InsufficientLiquidity();
```

```
        uint256 amountInWithFee = amountIn * (BPS - FEE_BPS);
        amountOut = (amountInWithFee * reserveOut) / (reserveIn * BPS + amountInWithFee);
    }

    /// @inheritdoc IPool
    function spotPriceE18() public view override returns (uint256 priceE18) {
        return _priceE18(_reserves.reserve0, _reserves.reserve1);
    }

    /// @inheritdoc IPool
    /// @dev Average price over [now - window, now]. The accumulator is interpolated to the exact window start
    ///      using the price that was in effect at that moment, so the result is not biased by older history.
    function consultTwapE18(uint32 window) external view override returns (uint256 priceE18) {
        if (window == 0) revert ZeroAmount();
        uint256 n = observations.length;
        if (n == 0 || block.timestamp < window) revert InsufficientHistory();
        uint256 target = block.timestamp - window;
        if (observations[0].timestamp > target) revert InsufficientHistory();

        // Binary search for the last observation at or before `target`.
        uint256 lo = 0;
        uint256 hi = n - 1;
        while (lo < hi) {
            uint256 mid = (lo + hi + 1) / 2;
            if (observations[mid].timestamp <= target) lo = mid;
            else hi = mid - 1;
        }

        Observation memory o = observations[lo];
        uint256 priceAfter;
        if (lo == n - 1) {
            priceAfter = _priceE18(_reserves.reserve0, _reserves.reserve1);
        } else {
            Observation memory next = observations[lo + 1];
            priceAfter = (uint256(next.cumulative) - o.cumulative) / (next.timestamp - o.timestamp);
        }
        uint256 cumulativeAtTarget = o.cumulative + priceAfter * (target - o.timestamp);
        uint256 cumulativeNow = priceCumulative + _priceE18(_reserves.reserve0, _reserves.reserve1) * (block.timestamp
- _reserves.timestamp);
        return (cumulativeNow - cumulativeAtTarget) / window;
    }

    // ------------------------------------------------------------ liquidity

    /// @inheritdoc IPool
    function addLiquidity(
        uint256 amount0Desired,
        uint256 amount1Desired,
        uint256 amount0Min,
        uint256 amount1Min,
        address to
    ) external override nonReentrant returns (uint256 amount0, uint256 amount1, uint256 shares) {
        if (amount0Desired == 0 || amount1Desired == 0) revert ZeroAmount();

        if (_reserves.reserve0 == 0 && _reserves.reserve1 == 0) {
            (amount0, amount1) = (amount0Desired, amount1Desired);
        } else {
            uint256 amount1Optimal = (amount0Desired * _reserves.reserve1) / _reserves.reserve0;
            if (amount1Optimal <= amount1Desired) {
                (amount0, amount1) = (amount0Desired, amount1Optimal);
            } else {
                (amount0, amount1) = ((amount1Desired * _reserves.reserve0) / _reserves.reserve1, amount1Desired);
            }
        }
        if (amount0 < amount0Min) revert SlippageExceeded(amount0, amount0Min);
        if (amount1 < amount1Min) revert SlippageExceeded(amount1, amount1Min);

        uint256 supply = totalSupply();
        if (supply == 0) {
            uint256 root = Math.sqrt(amount0 * amount1);
            if (root <= MINIMUM_LIQUIDITY) revert InsufficientLiquidityMinted();
            shares = root - MINIMUM_LIQUIDITY;
            _mint(DEAD, MINIMUM_LIQUIDITY);
        } else {
            shares = Math.min((amount0 * supply) / _reserves.reserve0, (amount1 * supply) / _reserves.reserve1);
        }
        if (shares == 0) revert InsufficientLiquidityMinted();

        _mint(to, shares);
        _update(_reserves.reserve0 + amount0, _reserves.reserve1 + amount1);

        IERC20(token0).safeTransferFrom(msg.sender, address(this), amount0);
        IERC20(token1).safeTransferFrom(msg.sender, address(this), amount1);
        emit LiquidityAdded(msg.sender, amount0, amount1, shares, to);
    }

```

```
    /// @inheritdoc IPool
    function removeLiquidity(
        uint256 shares,
        uint256 amount0Min,
        uint256 amount1Min,
        address to
    ) external override nonReentrant returns (uint256 amount0, uint256 amount1) {
        if (shares == 0) revert ZeroAmount();
        uint256 supply = totalSupply();
        amount0 = (shares * _reserves.reserve0) / supply;
        amount1 = (shares * _reserves.reserve1) / supply;
        if (amount0 == 0 || amount1 == 0) revert InsufficientLiquidity();
        if (amount0 < amount0Min) revert SlippageExceeded(amount0, amount0Min);
        if (amount1 < amount1Min) revert SlippageExceeded(amount1, amount1Min);

        _burn(msg.sender, shares);
        _update(_reserves.reserve0 - amount0, _reserves.reserve1 - amount1);

        IERC20(token0).safeTransfer(to, amount0);
        IERC20(token1).safeTransfer(to, amount1);
        emit LiquidityRemoved(msg.sender, amount0, amount1, shares, to);
    }

    // ------------------------------------------------------------------ swap

    /// @inheritdoc IPool
    function swapExactIn(
        address tokenIn,
        uint256 amountIn,
        uint256 minAmountOut,
        address to
    ) external override nonReentrant returns (uint256 amountOut) {
        amountOut = getAmountOut(tokenIn, amountIn);
        if (amountOut == 0) revert InsufficientLiquidity();
        if (amountOut < minAmountOut) revert SlippageExceeded(amountOut, minAmountOut);

        bool zeroForOne = tokenIn == token0;
        address tokenOut = zeroForOne ? token1 : token0;
        if (zeroForOne) _update(_reserves.reserve0 + amountIn, _reserves.reserve1 - amountOut);
        else _update(_reserves.reserve0 - amountOut, _reserves.reserve1 + amountIn);

        IERC20(tokenIn).safeTransferFrom(msg.sender, address(this), amountIn);
        IERC20(tokenOut).safeTransfer(to, amountOut);
        emit Swap(msg.sender, tokenIn, amountIn, amountOut, to);
    }

    // ------------------------------------------------------------------ internal

    /// @dev USDC per whole WBTC scaled by 1e18, floor division. Reverts if the pool is empty.
    function _priceE18(uint256 r0, uint256 r1) internal view returns (uint256) {
        if (r0 == 0 || r1 == 0) revert InsufficientLiquidity();
        return (r1 * priceScaleNum) / (r0 * priceScaleDen);
    }

    /// @dev Accumulates the price that was in effect since the last update, records an observation, then stores
    ///      the new reserves. Must be called before any token moves.
    function _update(uint256 newReserve0, uint256 newReserve1) internal {
        if (observations.length == 0) {
            observations.push(Observation(SafeCast.toUint32(block.timestamp), 0));
        } else {
            uint256 elapsed = block.timestamp - _reserves.timestamp;
            if (elapsed > 0) {
                priceCumulative += _priceE18(_reserves.reserve0, _reserves.reserve1) * elapsed;
                observations.push(
                    Observation(SafeCast.toUint32(block.timestamp), SafeCast.toUint224(priceCumulative))
                );
            }
        }
        _reserves = Reserves(
            SafeCast.toUint112(newReserve0),
            SafeCast.toUint112(newReserve1),
            SafeCast.toUint32(block.timestamp)
        );
    }
}
```

OrderBook.sol

ERC-721 streaming orders (SORDER) · deployed and verified on Sepolia at `0x9c20dbb0bb17a0532348e69502af3b8b44fefa62`

```
//SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

import { ERC721 } from "@openzeppelin/contracts/token/ERC721/ERC721.sol";
import { IERC20 } from "@openzeppelin/contracts/token/ERC20/IERC20.sol";
import { IERC20Metadata } from "@openzeppelin/contracts/token/ERC20/extensions/IERC20Metadata.sol";
import { SafeERC20 } from "@openzeppelin/contracts/token/ERC20/utils/SafeERC20.sol";
import { ReentrancyGuard } from "@openzeppelin/contracts/utils/ReentrancyGuard.sol";
import { SafeCast } from "@openzeppelin/contracts/utils/math/SafeCast.sol";
import { IOrderBook } from "./interfaces/IOrderBook.sol";
import { IPool } from "./interfaces/IPool.sol";
import { IPriceGuard } from "./interfaces/IPriceGuard.sol";
import { PriceMath } from "./libraries/PriceMath.sol";

/// @title OrderBook
/// @notice Streaming sell orders: a seller deposits WBTC and the order is swapped for USDC through one pool in a
///         fixed number of equal slices. Each order is an ERC-721; whoever holds the NFT owns the unsold WBTC and
///         the USDC proceeds, and can transfer, claim or cancel.
/// @dev There is no owner and no admin function: the pool and the price guard are fixed at deployment and nothing
///      else can be changed. Fee-on-transfer and rebasing tokens are not supported. The pool is trusted to pull
///      exactly `amountIn` and to return the real output amount.
contract OrderBook is IOrderBook, ERC721, ReentrancyGuard {
    using SafeERC20 for IERC20;

    /// @notice Upper limit on slices per order.
    uint256 public constant MAX_SLICES = 200;
    /// @notice Upper limit on the per-order tolerance below the oracle-implied output: 10%.
    uint256 public constant MAX_TOLERANCE_BPS = 1_000;
    uint256 internal constant BPS = 10_000;

    IPool public immutable pool;
    IPriceGuard public immutable priceGuard;
    IERC20 public immutable wbtc;
    IERC20 public immutable usdc;
    uint8 internal immutable wbtcDecimals;
    uint8 internal immutable usdcDecimals;

    /// @notice Id the next order will get. Ids start at 1.
    uint256 public nextOrderId = 1;

    /// @dev Storage layout of an order: two slots instead of the three that IOrderBook.Order would use.
    ///      slot 0: remainingIn | proceeds. slot 1: sliceSize (96 bits) | slicesLeft | interval | nextExecution |
    ///      toleranceBps. sliceSize is narrowed to 96 bits (about 7.9e28 raw units, 1e13 times the WBTC supply);
    ///      createOrder rejects anything larger.
    struct StoredOrder {
        uint128 remainingIn;
        uint128 proceeds;
        uint96 sliceSize;
        uint32 slicesLeft;
        uint32 interval;
        uint64 nextExecution;
        uint16 toleranceBps;
    }

    mapping(uint256 orderId => StoredOrder) internal _orders;

    constructor(IPool pool_, IPriceGuard priceGuard_) ERC721("Streaming Order", "SORDER") {
        pool = pool_;
        priceGuard = priceGuard_;
        IERC20 wbtc_ = IERC20(pool_.token0());
        IERC20 usdc_ = IERC20(pool_.token1());
        wbtc = wbtc_;
        usdc = usdc_;
        wbtcDecimals = IERC20Metadata(address(wbtc_)).decimals();
        usdcDecimals = IERC20Metadata(address(usdc_)).decimals();
    }

    /// @inheritdoc IOrderBook
    /// @dev Needs 1 to MAX_SLICES slices, at least 1 raw unit per slice, a non-zero interval and a tolerance of at
    ///      most MAX_TOLERANCE_BPS. The first slice can run `interval` seconds after creation. The tolerance must
    ///      leave room for the pool fee (30 bps) plus the expected price impact of one slice, or the slice reverts.
    function createOrder(
        uint256 totalIn,
        uint32 slices,
        uint32 interval,
        uint16 toleranceBps
    ) external override nonReentrant returns (uint256 orderId) {
        if (
            slices == 0 ||
            slices > MAX_SLICES ||
            totalIn < slices ||
            totalIn > type(uint128).max ||
```

```
            totalIn / slices > type(uint96).max ||
            interval == 0 ||
            toleranceBps > MAX_TOLERANCE_BPS
        ) revert InvalidParams();

        orderId = nextOrderId++;
        _orders[orderId] = StoredOrder({
            remainingIn: uint128(totalIn),
            proceeds: 0,
            sliceSize: uint96(totalIn / slices),
            slicesLeft: slices,
            interval: interval,
            nextExecution: SafeCast.toUint64(block.timestamp + interval),
            toleranceBps: toleranceBps
        });
        emit OrderCreated(orderId, msg.sender, totalIn, slices, interval, toleranceBps);

        wbtc.safeTransferFrom(msg.sender, address(this), totalIn);
        _safeMint(msg.sender, orderId);
    }

    /// @inheritdoc IOrderBook
    /// @dev Anyone may call (the keeper is permissionless). Reverts OrderFinished when no slices are left and
    ///      TooEarly before `nextExecution`. Reverts if the Chainlink feed is bad (see PriceGuard). If the pool
    ///      price is outside the guard bounds the slice is skipped: SliceSkipped is emitted, no tokens move and the
    ///      next attempt is `interval` seconds later. Otherwise slice size is totalIn / slices, except the last
    ///      slice, which sells everything left so rounding dust is never stranded. The pool call must return at
    ///      least the oracle-implied amount minus the tolerance, or the whole call reverts. After each attempt the
    ///      next slice is due `interval` seconds from now (not from the previous due time), so a late keeper never
    ///      causes a burst of back-to-back slices.
    function executeSlice(uint256 orderId) external override nonReentrant returns (bool executed) {
        _requireOwned(orderId);
        StoredOrder storage order = _orders[orderId];
        if (order.slicesLeft == 0) revert OrderFinished();
        if (block.timestamp < order.nextExecution) revert TooEarly(order.nextExecution);

        uint256 poolPriceE18 = pool.spotPriceE18();
        if (!priceGuard.poolWithinBounds(poolPriceE18)) {
            order.nextExecution = SafeCast.toUint64(block.timestamp + order.interval);
            emit SliceSkipped(orderId, poolPriceE18, priceGuard.getOraclePrice());
            return false;
        }

        uint256 oraclePriceE18 = priceGuard.getOraclePrice();
        uint256 amountIn = order.slicesLeft == 1 ? order.remainingIn : order.sliceSize;
        uint256 minAmountOut = (PriceMath.quoteRaw(amountIn, oraclePriceE18, wbtcDecimals, usdcDecimals) *
            (BPS - order.toleranceBps)) / BPS;

        order.remainingIn -= uint128(amountIn);
        order.slicesLeft -= 1;
        order.nextExecution = SafeCast.toUint64(block.timestamp + order.interval);

        wbtc.forceApprove(address(pool), amountIn);
        uint256 amountOut = pool.swapExactIn(address(wbtc), amountIn, minAmountOut, address(this));
        order.proceeds += SafeCast.toUint128(amountOut);

        emit SliceExecuted(orderId, amountIn, amountOut, order.slicesLeft);
        return true;
    }

    /// @inheritdoc IOrderBook
    /// @dev Anyone may call; the USDC always goes to the current `ownerOf(orderId)`, so a transferred order pays
    ///      its new holder. Reverts NothingToClaim when there are no proceeds.
    function claim(uint256 orderId) external override nonReentrant returns (uint256 amountOut) {
        address holder = ownerOf(orderId);
        amountOut = _orders[orderId].proceeds;
        if (amountOut == 0) revert NothingToClaim();
        _orders[orderId].proceeds = 0;
        usdc.safeTransfer(holder, amountOut);
        emit Claimed(orderId, holder, amountOut);
    }

    /// @inheritdoc IOrderBook
    /// @dev Only the NFT holder (not an approved operator) may cancel. Pays out the unsold WBTC and any unclaimed
    ///      USDC, deletes the order and burns the NFT. Works on a finished order too, as claim-and-burn.
    function cancel(uint256 orderId) external override nonReentrant {
        address holder = ownerOf(orderId);
        if (msg.sender != holder) revert NotOrderOwner();
        uint256 refundedIn = _orders[orderId].remainingIn;
        uint256 proceedsPaid = _orders[orderId].proceeds;
        delete _orders[orderId];
        _burn(orderId);

        if (refundedIn != 0) wbtc.safeTransfer(holder, refundedIn);
        if (proceedsPaid != 0) usdc.safeTransfer(holder, proceedsPaid);
        emit Cancelled(orderId, holder, refundedIn, proceedsPaid);
```

```
    }

    /// @inheritdoc IOrderBook
    function getOrder(uint256 orderId) external view override returns (Order memory) {
        _requireOwned(orderId);
        StoredOrder memory o = _orders[orderId];
        return
            Order({
                remainingIn: o.remainingIn,
                proceeds: o.proceeds,
                sliceSize: o.sliceSize,
                slicesLeft: o.slicesLeft,
                interval: o.interval,
                nextExecution: o.nextExecution,
                toleranceBps: o.toleranceBps
            });
    }
}
```

PriceGuard.sol

Chainlink checks and pool deviation band · deployed and verified on Sepolia at `0xe52d276bec7b96bcdd0632050cad5717956289df`

```
//SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

import { Ownable } from "@openzeppelin/contracts/access/Ownable.sol";
import { AggregatorV3Interface } from "@chainlink/contracts/src/v0.8/shared/interfaces/AggregatorV3Interface.sol";
import { IPriceGuard } from "./interfaces/IPriceGuard.sol";
import { PriceMath } from "./libraries/PriceMath.sol";

/// @title PriceGuard
/// @notice Reads a Chainlink BTC/USD feed and tells the OrderBook whether the pool price is close enough to it.
/// @dev The owner can change exactly two things, each with a hard cap and an event: `maxStaleness` (1 second up to
///      MAX_STALENESS_CAP) and `maxDeviationBps` (1 bp up to MAX_DEVIATION_BPS_CAP). The owner cannot change the
///      feed, pause the guard or move funds; the guard holds no funds.
contract PriceGuard is IPriceGuard, Ownable {
    /// @notice Hard upper limit for `maxStaleness`: one day.
    uint256 public constant MAX_STALENESS_CAP = 1 days;
    /// @notice Hard upper limit for `maxDeviationBps`: 20%.
    uint256 public constant MAX_DEVIATION_BPS_CAP = 2_000;
    uint256 internal constant BPS = 10_000;

    AggregatorV3Interface public immutable feed;
    uint8 public immutable feedDecimals;

    uint256 public override maxStaleness;
    uint256 public override maxDeviationBps;

    error ZeroAddress();
    error ZeroValue();

    constructor(
        AggregatorV3Interface feed_,
        address initialOwner,
        uint256 maxStaleness_,
        uint256 maxDeviationBps_
    ) Ownable(initialOwner) {
        if (address(feed_) == address(0)) revert ZeroAddress();
        feed = feed_;
        feedDecimals = feed_.decimals();
        _setMaxStaleness(maxStaleness_);
        _setMaxDeviationBps(maxDeviationBps_);
    }

    /// @inheritdoc IPriceGuard
    /// @dev Reverts InvalidPrice on answer <= 0, StalePrice when the round is older than `maxStaleness` (or has no
    ///      timestamp, or is dated in the future), and StaleRound when answeredInRound < roundId.
    function getOraclePrice() public view override returns (uint256 priceE18) {
        (uint80 roundId, int256 answer, , uint256 updatedAt, uint80 answeredInRound) = feed.latestRoundData();
        if (answer <= 0) revert InvalidPrice(answer);
        if (updatedAt == 0 || updatedAt > block.timestamp || block.timestamp - updatedAt > maxStaleness) {
            revert StalePrice(updatedAt, maxStaleness);
        }
        if (answeredInRound < roundId) revert StaleRound(roundId, answeredInRound);
        return PriceMath.feedToE18(answer, feedDecimals);
    }

    /// @inheritdoc IPriceGuard
    /// @dev |pool - oracle| * 10_000 <= oracle * maxDeviationBps, so exactly at the limit still counts as within.
    function poolWithinBounds(uint256 poolPriceE18) external view override returns (bool) {
        uint256 oracle = getOraclePrice();
        uint256 diff = poolPriceE18 > oracle ? poolPriceE18 - oracle : oracle - poolPriceE18;
        return diff * BPS <= oracle * maxDeviationBps;
    }

    /// @inheritdoc IPriceGuard
    /// @dev Owner only. Range 1 to MAX_STALENESS_CAP seconds.
    function setMaxStaleness(uint256 newMaxStaleness) external override onlyOwner {
        _setMaxStaleness(newMaxStaleness);
    }

    /// @inheritdoc IPriceGuard
    /// @dev Owner only. Range 1 to MAX_DEVIATION_BPS_CAP.
    function setMaxDeviationBps(uint256 newMaxDeviationBps) external override onlyOwner {
        _setMaxDeviationBps(newMaxDeviationBps);
    }

    function _setMaxStaleness(uint256 value) internal {
        if (value == 0) revert ZeroValue();
        if (value > MAX_STALENESS_CAP) revert StalenessAboveCap(value, MAX_STALENESS_CAP);
        maxStaleness = value;
        emit MaxStalenessSet(value);
    }

```

```
    function _setMaxDeviationBps(uint256 value) internal {
        if (value == 0) revert ZeroValue();
        if (value > MAX_DEVIATION_BPS_CAP) revert DeviationAboveCap(value, MAX_DEVIATION_BPS_CAP);
        maxDeviationBps = value;
        emit MaxDeviationBpsSet(value);
    }
}
```

libraries/PriceMath.sol

All price and amount conversions in one place

```
1  //SPDX-License-Identifier: MIT
2  pragma solidity ^0.8.20;
3
4  import { Math } from "@openzeppelin/contracts/utils/math/Math.sol";
5
6  /// @title PriceMath
7  /// @notice The only place where oracle decimals and token decimals are converted on the order side.
8  /// @dev Project price unit: USDC per ONE WHOLE WBTC, scaled by 1e18 (named `priceE18`). Integer maths with floor
9  ///      division so the Python notebook can reproduce every result exactly. USDC is treated as exactly 1 USD,
10 ///      which is how a BTC/USD feed is turned into a WBTC/USDC price.
11 library PriceMath {
12     uint256 internal constant E18 = 1e18;
13
14     /// @notice Converts a Chainlink answer with `feedDecimals` decimals into priceE18.
15     /// @dev Caller must have checked `answer > 0`. Feed decimals above 18 round down.
16     function feedToE18(int256 answer, uint8 feedDecimals) internal pure returns (uint256) {
17         uint256 a = uint256(answer);
18         if (feedDecimals <= 18) return a * 10 ** (18 - feedDecimals);
19         return a / 10 ** (feedDecimals - 18);
20     }
21
22     /// @notice Raw amount of the quote token (USDC, `decimalsOut`) received for `amountIn` raw units of the base
23     ///         token (WBTC, `decimalsIn`) at `priceE18`, before any fee or price impact.
24     /// @dev amountOut = amountIn * priceE18 * 10^decimalsOut / (10^decimalsIn * 1e18), rounded down.
25     function quoteRaw(
26         uint256 amountIn,
27         uint256 priceE18,
28         uint8 decimalsIn,
29         uint8 decimalsOut
30     ) internal pure returns (uint256) {
31         return Math.mulDiv(amountIn * priceE18, 10 ** decimalsOut, 10 ** decimalsIn * E18);
32     }
33 }
```

interfaces/IPool.sol

Interface (frozen)

```
//SPDX-License-Identifier: MIT
pragma solidity >=0.8.0 <0.9.0;

/// @title IPool
/// @notice Interface the OrderBook and PriceGuard build against.
/// @dev STATUS: approved by Dev; waiting for Aditya's ack in the group chat before the day-two freeze.
///      token0 = WBTC (8 decimals), token1 = USDC (6 decimals); the pool reads both decimals at construction.
///      PRICE UNIT (one unit across Pool, PriceGuard and OrderBook): USDC per ONE WHOLE WBTC, scaled by 1e18,
///      so 60_000 USDC/BTC is 60_000e18. Raw-unit ratios never leave the Pool.
interface IPool {
    event LiquidityAdded(address indexed provider, uint256 amount0, uint256 amount1, uint256 shares, address indexed
to);
    event LiquidityRemoved(address indexed provider, uint256 amount0, uint256 amount1, uint256 shares, address indexed
to);
    event Swap(address indexed sender, address indexed tokenIn, uint256 amountIn, uint256 amountOut, address indexed
to);


    error ZeroAmount();
    error InvalidToken();
    error InsufficientLiquidity();
    error InsufficientLiquidityMinted();
    error SlippageExceeded(uint256 amountOut, uint256 minAmountOut);
    error InsufficientHistory();

    function token0() external view returns (address);

    function token1() external view returns (address);

    /// @return reserve0 token0 reserve
    /// @return reserve1 token1 reserve
    /// @return blockTimestampLast timestamp of the last reserve update
    function getReserves() external view returns (uint112 reserve0, uint112 reserve1, uint32 blockTimestampLast);

    /// @notice Pulls tokens from msg.sender (needs prior approval) and mints LP shares to `to`.
    /// @dev Amounts actually taken may be below the desired amounts, never below the minimums.
    function addLiquidity(
        uint256 amount0Desired,
        uint256 amount1Desired,
        uint256 amount0Min,
        uint256 amount1Min,
        address to
    ) external returns (uint256 amount0, uint256 amount1, uint256 shares);

    /// @notice Burns msg.sender's LP shares and sends the underlying tokens to `to`.
    function removeLiquidity(
        uint256 shares,
        uint256 amount0Min,
        uint256 amount1Min,
        address to
    ) external returns (uint256 amount0, uint256 amount1);

    /// @notice Pulls `amountIn` of `tokenIn` from msg.sender and sends the other token to `to`.
    /// @dev Reverts with SlippageExceeded if the output is below `minAmountOut`.
    function swapExactIn(
        address tokenIn,
        uint256 amountIn,
        uint256 minAmountOut,
        address to
    ) external returns (uint256 amountOut);

    /// @notice Quote for swapExactIn with the 30 bps fee applied.
    function getAmountOut(address tokenIn, uint256 amountIn) external view returns (uint256 amountOut);

    /// @notice Swap fee in basis points (immutable, 30).
    function FEE_BPS() external view returns (uint256);

    /// @notice Spot price in USDC per whole WBTC, scaled by 1e18, derived from the current reserves.
    function spotPriceE18() external view returns (uint256 priceE18);

    /// @notice Time-weighted average price over the last `window` seconds, in the same unit as spotPriceE18.
    /// @dev Reverts with InsufficientHistory if the pool is younger than `window`.
    function consultTwapE18(uint32 window) external view returns (uint256 priceE18);
}
```

interfaces/IOrderBook.sol

Interface (frozen)

```
//SPDX-License-Identifier: MIT
pragma solidity >=0.8.0 <0.9.0;

/// @title IOrderBook
/// @notice Streaming sell orders, one ERC-721 per order. The NFT holder owns the unfilled balance and proceeds.
/// @dev STATUS: approved by Dev; waiting for Aditya's ack before the day-two freeze.
///      Orders sell WBTC for USDC through the single pool fixed at construction.
interface IOrderBook {
    struct Order {
        uint128 remainingIn; // WBTC not yet swapped
        uint128 proceeds; // USDC swapped but not yet claimed
        uint128 sliceSize; // totalIn / slices; the last slice also takes the rounding remainder
        uint32 slicesLeft;
        uint32 interval; // seconds between slices
        uint64 nextExecution; // earliest timestamp for the next slice
        uint16 toleranceBps; // allowed shortfall of amountOut versus the oracle-implied amount
    }

    event OrderCreated(
        uint256 indexed orderId,
        address indexed owner,
        uint256 totalIn,
        uint32 slices,
        uint32 interval,
        uint16 toleranceBps
    );
    event SliceExecuted(uint256 indexed orderId, uint256 amountIn, uint256 amountOut, uint32 slicesLeft);
    event SliceSkipped(uint256 indexed orderId, uint256 poolPriceE18, uint256 oraclePriceE18);
    event Claimed(uint256 indexed orderId, address indexed to, uint256 amountOut);
    event Cancelled(uint256 indexed orderId, address indexed to, uint256 refundedIn, uint256 proceedsPaid);

    error InvalidParams();
    error NotOrderOwner();
    error TooEarly(uint256 nextExecution);
    error OrderFinished();
    error NothingToClaim();

    /// @notice Pulls `totalIn` WBTC from msg.sender (needs approval) and mints the order NFT to them.
    function createOrder(
        uint256 totalIn,
        uint32 slices,
        uint32 interval,
        uint16 toleranceBps
    ) external returns (uint256 orderId);

    /// @notice Callable by anyone once `nextExecution` has passed. Swaps one slice through the pool, or
    ///         emits SliceSkipped and returns false if PriceGuard says the pool is out of bounds.
    function executeSlice(uint256 orderId) external returns (bool executed);

    /// @notice Pays accumulated USDC proceeds to the current ownerOf(orderId).
    function claim(uint256 orderId) external returns (uint256 amountOut);

    /// @notice Owner only. Returns the unfilled WBTC and any unclaimed USDC to the owner and burns the NFT.
    function cancel(uint256 orderId) external;

    /// @notice Returns the stored order state.
    function getOrder(uint256 orderId) external view returns (Order memory);
}
```

interfaces/IPriceGuard.sol

Interface (frozen)

```
//SPDX-License-Identifier: MIT
pragma solidity >=0.8.0 <0.9.0;

/// @title IPriceGuard
/// @notice Chainlink BTC/USD reader and pool-vs-oracle deviation check used by the OrderBook.
/// @dev STATUS: approved by Dev with the unit change below; waiting for Aditya's ack before the day-two freeze.
///      Both prices use the single project unit: USDC per whole WBTC, scaled by 1e18.
interface IPriceGuard {
    event MaxStalenessSet(uint256 maxStaleness);
    event MaxDeviationBpsSet(uint256 maxDeviationBps);

    error InvalidPrice(int256 answer);
    error StalePrice(uint256 updatedAt, uint256 maxStaleness);
    error StaleRound(uint80 roundId, uint80 answeredInRound);
    error StalenessAboveCap(uint256 value, uint256 cap);
    error DeviationAboveCap(uint256 value, uint256 cap);

    /// @notice Oracle price as USDC per WBTC in whole units, scaled by 1e18 (60_000 USDC/BTC = 60_000e18).
    /// @dev Reverts on answer <= 0, on stale updatedAt, and when answeredInRound < roundId.
    function getOraclePrice() external view returns (uint256 priceE18);

    /// @notice True if `poolPrice` is within the configured deviation of the oracle.
    /// @param poolPriceE18 Pool price from IPool.spotPriceE18(), same unit as getOraclePrice().
    /// @dev Reverts like getOraclePrice() when the feed is bad; returns false (never reverts) on deviation.
    function poolWithinBounds(uint256 poolPriceE18) external view returns (bool);

    function maxStaleness() external view returns (uint256);

    function maxDeviationBps() external view returns (uint256);

    /// @dev Owner only, reverts above a hard cap.
    function setMaxStaleness(uint256 newMaxStaleness) external;

    /// @dev Owner only, reverts above a hard cap.
    function setMaxDeviationBps(uint256 newMaxDeviationBps) external;
}
```

mocks/MockWBTC.sol

Test WBTC, 8 decimals, capped faucet · deployed and verified on Sepolia at `0x2c95a78b3a3fc73c16bdd6394bfe30cc51d37d12`

```sol
//SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

import { ERC20 } from "@openzeppelin/contracts/token/ERC20/ERC20.sol";
import { Ownable } from "@openzeppelin/contracts/access/Ownable.sol";

/// @notice Test WBTC with 8 decimals. Owner can mint any amount; anyone can use the capped faucet.
contract MockWBTC is ERC20, Ownable {
    /// @notice Maximum amount a single faucet call can mint (5 WBTC).
    uint256 public constant FAUCET_CAP = 5e8;

    error FaucetCapExceeded(uint256 amount, uint256 cap);

    constructor(address initialOwner) ERC20("Mock Wrapped BTC", "mWBTC") Ownable(initialOwner) {}

    /// @notice Token decimals, fixed by the real asset this mock stands in for.
    function decimals() public pure override returns (uint8) {
        return 8;
    }

    /// @notice Mint `amount` (raw units) to `to`. Only the owner can call this.
    /// @param to Recipient of the new tokens.
    /// @param amount Amount in raw token units.
    function mint(address to, uint256 amount) external onlyOwner {
        _mint(to, amount);
    }

    /// @notice Anyone can mint up to `FAUCET_CAP` (raw units) to themselves, for Sepolia testing.
    /// @param amount Amount in raw token units; reverts with FaucetCapExceeded above the cap.
    function faucet(uint256 amount) external {
        if (amount > FAUCET_CAP) revert FaucetCapExceeded(amount, FAUCET_CAP);
        _mint(msg.sender, amount);
    }
}
```

mocks/MockUSDC.sol

Test USDC, 6 decimals, capped faucet · deployed and verified on Sepolia at `0x9cd5587b22d15790a5880d37837cdd091c9ea0b5`

```
//SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

import { ERC20 } from "@openzeppelin/contracts/token/ERC20/ERC20.sol";
import { Ownable } from "@openzeppelin/contracts/access/Ownable.sol";

/// @notice Test USDC with 6 decimals. Owner can mint any amount; anyone can use the capped faucet.
contract MockUSDC is ERC20, Ownable {
    /// @notice Maximum amount a single faucet call can mint (500,000 USDC).
    uint256 public constant FAUCET_CAP = 500_000e6;

    error FaucetCapExceeded(uint256 amount, uint256 cap);

    constructor(address initialOwner) ERC20("Mock USD Coin", "mUSDC") Ownable(initialOwner) {}

    /// @notice Token decimals, fixed by the real asset this mock stands in for.
    function decimals() public pure override returns (uint8) {
        return 6;
    }

    /// @notice Mint `amount` (raw units) to `to`. Only the owner can call this.
    /// @param to Recipient of the new tokens.
    /// @param amount Amount in raw token units.
    function mint(address to, uint256 amount) external onlyOwner {
        _mint(to, amount);
    }

    /// @notice Anyone can mint up to `FAUCET_CAP` (raw units) to themselves, for Sepolia testing.
    /// @param amount Amount in raw token units; reverts with FaucetCapExceeded above the cap.
    function faucet(uint256 amount) external {
        if (amount > FAUCET_CAP) revert FaucetCapExceeded(amount, FAUCET_CAP);
        _mint(msg.sender, amount);
    }
}
```